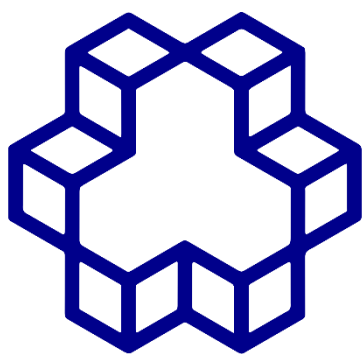




دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق - گروه مهندسی کنترل

مبانی سیستم‌های هوشمند

مینی پروژه دوم

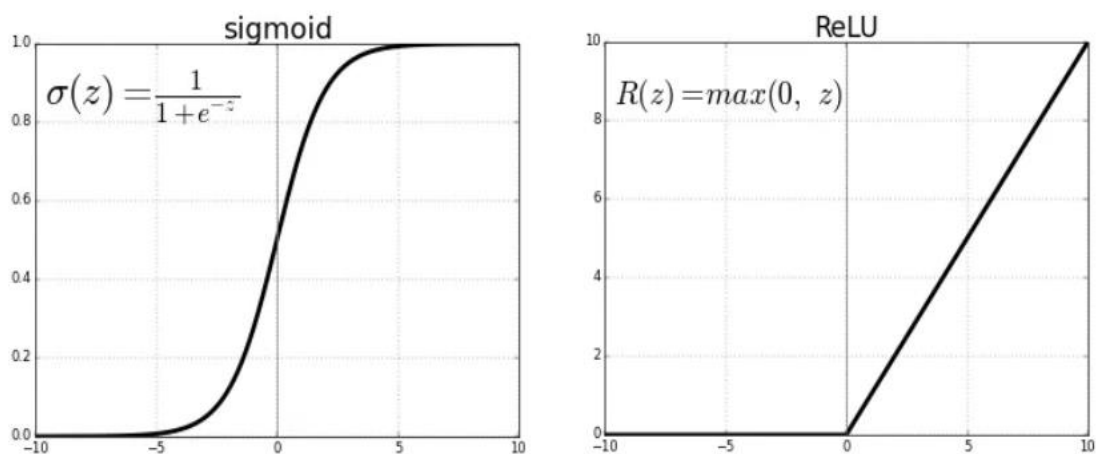
نام و نام خانوادگی	مهدی فیاض
شماره دانشجویی	۴۰۰۰۷۹۱۳
تاریخ	۱۴۰۳/۱۰/۱۰

۱ پرسش یک

۱

فرض کنید در یک مسئله طبقه بندی دو کلاسه، دو لایه انتهایی شبکه شما فعال ساز ReLU و سیگموید است. چه اتفاقی می افتد؟ این ترکیب در دو لایه انتهایی شبکه برای یک مسئله طبقه بندی دو کلاسه مناسب نیست، زیرا خروجی ReLU برای مقداری منفی صفر است و لذا مقدار وروی سیگموید ما هیچ زمانی مقدار منفی نمی گیرد و همه ی خروجی های نهایی در بازه ی (0.5 - 1) هستند و به این طریق نمی توان تمایزی بین دو کلاس قائل شد.

گرادیان در ناحیه منفی برای ReLU صفر است و این امر منجر به «مرده شدن نرون ها» می شود. (به اصطلاح 😊)



۲

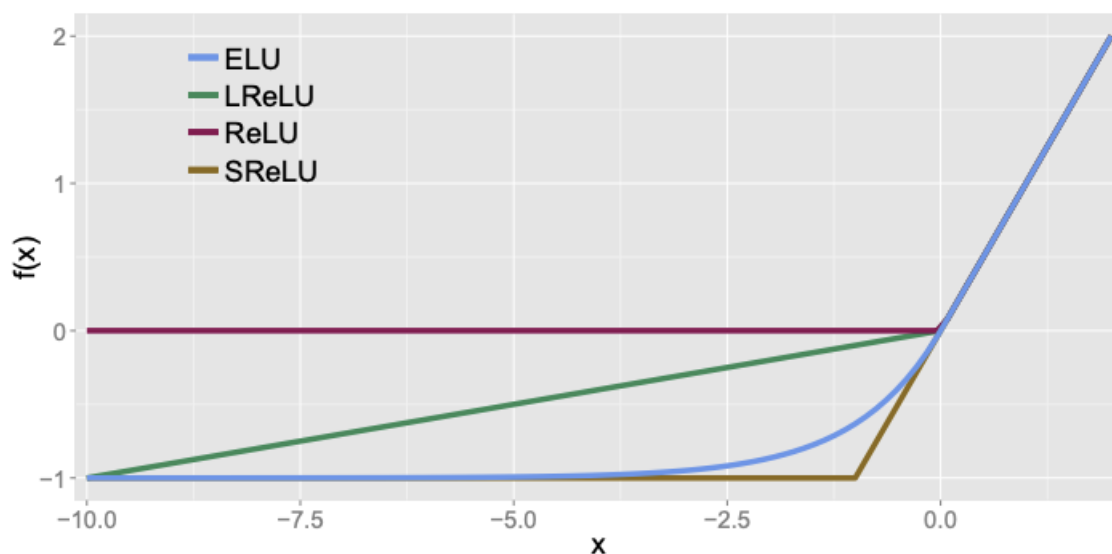
یک جایگزین برای ReLU معادله ۱ آورده شده است. ضمن محاسبه گرادیان آن، حداقل یک مزیت آن نسبت

به ReLU را توضیح دهید.

Function	Derivative
$R(z) = \begin{cases} z & z > 0 \\ \alpha \cdot (e^z - 1) & z \leq 0 \end{cases}$	$R'(z) = \begin{cases} 1 & z > 0 \\ \alpha \cdot e^z & z < 0 \end{cases}$

همانطور که مشاهده می‌شود، بر خلاف RELU، مشتق (گرادیان) این تابع هیچ گاه صفر نمی‌شود. در ELU، به دلیل وجود جزء نمایی برای مقادیر منفی، گرادیان همیشه مثبت است و به صفر میل نمی‌کند. این ویژگی باعث می‌شود که نورون‌ها کمتر در حالت غیرفعال (مرده) قرار بگیرند و به طور فعال در فرآیند یادگیری شرکت کنند. (برخلاف RELU مقادیر منفی که مربوط به یکی از کلاس‌ها بودند را صفر نمی‌کند)

همچنین یکی دیگر از مزیت‌های این تابع فعال‌سازی این است که ELU خروجی متوسطی نزدیک به صفر دارد که می‌تواند به بهبود عملکرد مدل‌های مولد (Generative Models) کمک کند.



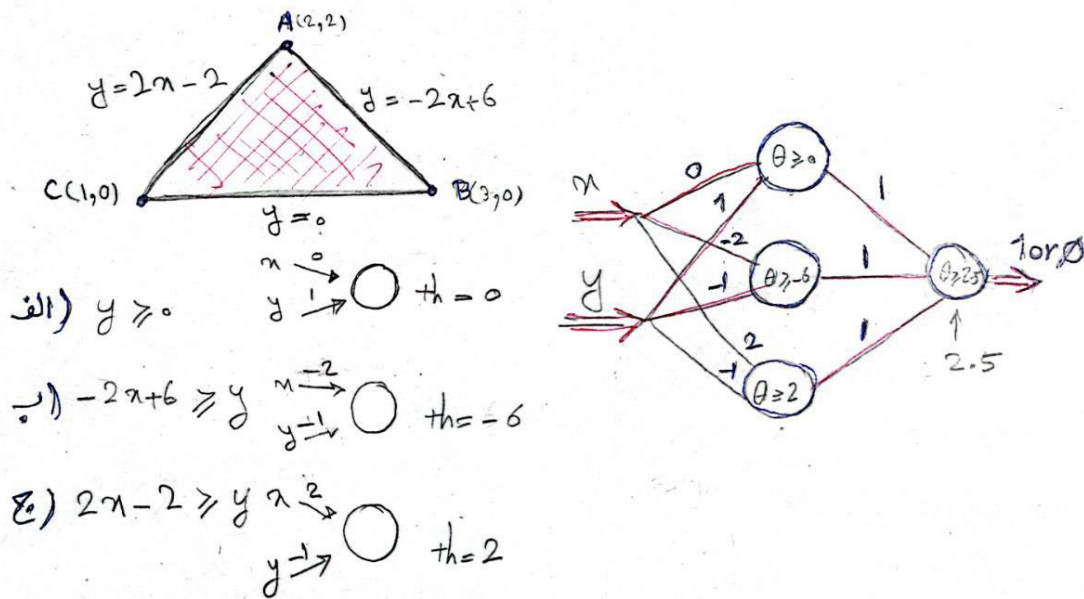
۳

برای تشخیص نقاط داخل مثلث می‌توان از یک شبکه دو لایه استفاده کرد. در لایه اول سه نورون قرار می‌دهیم که وظیفه‌ی آن‌ها است که تعیین کنند نقطه‌ی ورودی نسبت به هر کدام از اضلاع در چه موقعیتی قرار دارند. در لایه انتهایی نیز یک نورون قرار می‌دهیم تا اگر خروجی‌های سه نورون لایه قبل به گونه‌ی دلخواه (داخل مثلث) بود مقدار یک و در غیر اینصورت مقدار صفر را خروجی بدهد.

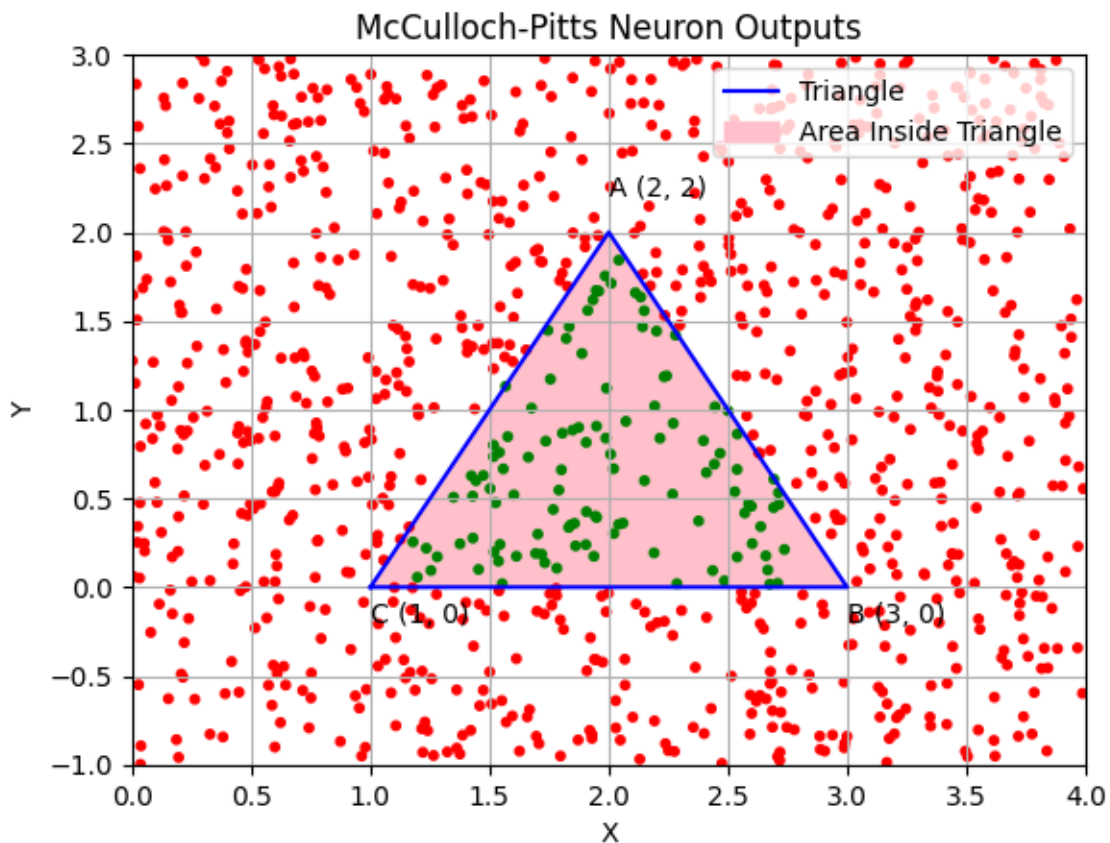
خطی که از $A(1,0)$ و $B(3,0)$ عبور می‌کند: $y=0$

خطی که از $C(2,2)$ و $B(3,0)$ عبور می‌کند: $y = -2x + 6$

خطی که از $A(1,0)$ و $C(2,2)$ عبور می‌کند: $y = 2x - 2$



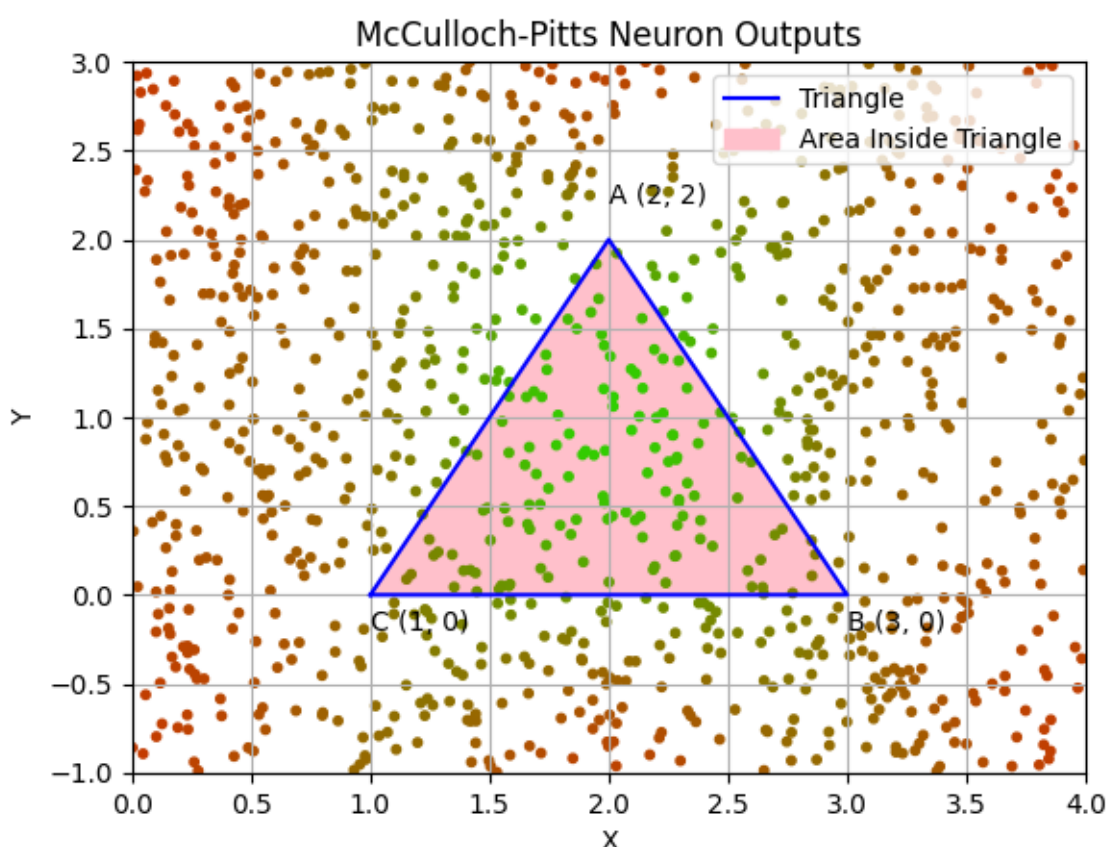
کد برنامه در فایل colab آورده شده است. با یک نورون ساده که صرفاً دارای ضرب باشد نمی‌توان این کار را انجام داد اما با نورون McCulloch-Pitts که دارای آستانه است و با پرسپترون که دارای تابع فعال ساز است می‌توان این کار را انجام داد. می‌توان گفت McCulloch-Pitts همان پرسپترون است با تابع فعال ساز پله و بایاسی که قرینه آستانه است. هر دو نورون پیاده سازی شده و تست شدند.



شکل با هم با نرون پرسپترون و هم McCulloch پیاده سازی شد.

این کار را با استفاده از تابع های فعال سازی نیز انجام دادیم.

به گونه ای که در لایه اول از توزیع نرمال با میانگین یک و واریانس یک استفاده کردیم با این توجیه که بیشتر مقدار ها به نقاطی که در سمت درست خط و فاصله یک از آن قرار دارند تعلق می گیرد و با واریانس یک این مقدار به صفر میل می کند. همچنین در نرون خروجی نیز یک تابع فعال ساز سیگنوید قرار داده تا مقادیر هر چه مثبت تر بودند به یک نزدیک تر و رنگ سبز بگیرند و مقادیر منفی رنگ قرمز و اعداد بینابین، طیف رنگی میان سبز و قرمز و خروجی شکل زیر شد:



داغون شدیم آقا از بس این ضرایبو تغییر دادیم و منت گرادیان نکشیدیم. 😞

تایتل شکل دوم اشتباهه، پرسپترونه آقا. نه مک کلا یه چیزی.

۲ پرسش دو

۱

کد مربوطه در فایل کولب آورده شده است.

۲

همبستگی داده ها با هدف بسیار کم است و این کار مدل سازی را سخت می کند. 😞

۳

کد مربوطه در فایل کولب آورده شده است. ۷۰ درصد به داده آموزش، ۲۰ درصد به داده تست و ۱۰ درصد به داده اعتبار سنجی اختصاص یافت. از ویژگی stratify نیز استفاده شد تا به نسبت یکسان از کلاس ها در داده های آموزش و تست قرار بگیرد. ۱۰۰۰ نمونه با ۱۱ ویژگی داریم که تعداد کمی داده حساب می شود برای این مسئله 😞.

۴

حالت الف)

در ابتدا، هم برای مدل تک لایه و هم برای مدل دو لایه با تعداد نرون ۲ و ۴ و ۸ و ۱۶ و ۳۲ و ۶۴ مدل ها را آموزش و میزان دقت را مشاهده کردیم. سپس نرمال سازی را نیز در هر لایه انجام دادیم. در مدل تک لایه تعداد ۱۶ نرون بهترین و در مدل ۲ لایه تعداد ۸*۸ نرون بهترین مدل از بین مدل های انتخاب شده بود.

این دو مدل را انتخاب کرده و در مراحل بعد به ترتیب دراپ اون و رگولاریزیشن را به مدل هایمان اضافه کردیم. مشاهده شد که دقت در این دو مرحله از مراحل قبل بیشتر بود. همچنین بهینه ساز های خواسته شده را نیز به مدل اعمال کرده و نتایج را مشاهده کردیم.

البته نتایج دو پارگراف بالا اصلا قابل اعتماد نیست، چرا که در عین حال که random seed را در جای جای کد وارد کردم ولی باز هم با هر بار اجرای مدل، نتیجه ی مختلفی می گرفتم و مقایسه بین مدل ها در اون حالت درست نبود.

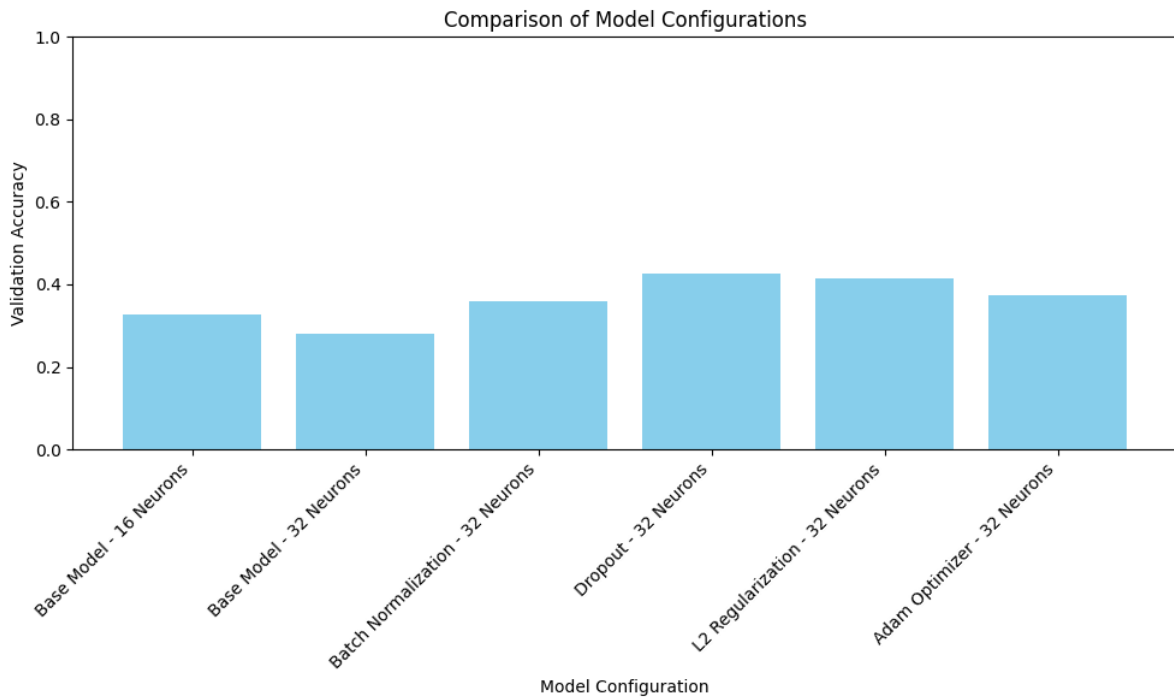
نتایج ارزیابی مدل ها:

	model	config	val_acc
0	1layer_16	base	0.413333
1	1layer_16	batch_norm	0.400000
2	1layer_16	dropout	0.320000
3	1layer_16	12	0.313333
4	1layer_16	adam	0.433333
5	1layer_32	base	0.400000
6	1layer_32	batch_norm	0.420000
7	1layer_32	dropout	0.346667
8	1layer_32	12	0.346667
9	1layer_32	adam	0.380000
10	2layers_(16, 8)	base	0.393333
11	2layers_(16, 8)	batch_norm	0.373333
12	2layers_(16, 8)	dropout	0.313333
13	2layers_(16, 8)	12	0.320000
14	2layers_(16, 8)	adam	0.413333
15	2layers_(32, 16)	base	0.360000
16	2layers_(32, 16)	batch_norm	0.420000
17	2layers_(32, 16)	dropout	0.426667
18	2layers_(32, 16)	12	0.386667
19	2layers_(32, 16)	adam	0.433333

1 با پیکربندی 1layer_16 بهترین مدل: 0.4333
دقت اعتبارسنجی: 0.4333

حالت ب)

با اضافه کردن ویژگی‌های مربوطه مشاهده شد که دقت کاهش می‌یابد. به علت اینکه دیتاهای موجود به اندازه‌ی کافی زیاد نیستند این کاهش دقت اتفاق می‌افتد.



البته اگر به صورت پشت سر هم ویژگی‌ها را اضافه کنیم دقت افزوده می‌شود اما دونه دونه برعکس! نمیدونم چطو داره

مشه: /

۵

کد در فایل آورده شده است. ۱۰ نمونه دیتای تست به صورت رندوم انتخاب شده و به مدل‌های تک لایه و دو لایه دادیم و مقادیر پیش‌بینی شده را با مقادیر اصلی مقایسه کرده و در یک جدول آوردیم.

```
Test Accuracy (1 layer, 16 neurons): 0.36666667461395264
Test Accuracy (2 layers, 8*8 neurons): 0.37142857909202576
1/1 _____ 0s 66ms/step
1/1 _____ 0s 86ms/step
Sample | True Label | Model 1 Prediction | Model 2 Prediction
1       | 4          | 3                  | 1
2       | 3          | 1                  | 2
3       | 4          | 4                  | 4
4       | 2          | 2                  | 3
5       | 3          | 1                  | 3
6       | 1          | 1                  | 1
7       | 4          | 1                  | 1
8       | 1          | 3                  | 3
9       | 2          | 2                  | 3
10      | 1          | 1                  | 1
```

دو روش برای ترکیب جواب مدل ها:

۱: Ensemble Averaging (میانگین گیری): خروجی های احتمالاتی مدل ها را محاسبه کرده و میانگین آن ها را

می گیریم.

Weighted Voting (رای گیری وزنی): به خروجی هر مدل وزن متناسب با عملکرد آن در اعتبارسنجی می دهیم و

ترکیب می کنیم.

یک روش دیگر برای این کار الگوریتم K_FOLD است که مثلاً دیتا ها را به سه بخش تقسیم کرده و برای هر مدل یک قسمت را به عنوان test می دهیم. برای predict کردن هم به هر سه مدل داده را می دهیم و رای ۲ از ۳ معتبر است.

با اعمال روش Ensemble دیدیم که دقت حدود ۵۰ درصد افزوده شده و از حدود ۴۰ به ۶۰ درصد رسید. در واقع نتایج پیش بینی مدل ها به نوعی با یکدیگر ترکیب شده و به هم کمک می کنند.

۳ پرسش سه

۱

توضیح تابع convertImageToBinary

فانکشن اول همانطور که از اسمش پیداست (convertImageToBinary) مسیر عکس را دریافت کرده و در نهایت یک لیست باینری از پیکس ها را برمیگرداند. توضیح خط به خط تابع در زیر آمده:

```
from PIL import Image, ImageDraw
import random
```

در ابتدا کتابخانه های مربوط به پردازش تصویر و تولید عدد رندوم اضافه می شوند.

```
def convertImageToBinary(path):
    """
    Convert an image to a binary representation based on pixel
    intensity.
    Args:
        path (str): The file path to the input image.
    Returns:
        list: A binary representation of the image where white is
        represented by -1 and black is represented by 1.
    """
```

```
image = Image.open(path)
```

```
draw = ImageDraw.Draw(image)
```

سپس تابع تعریف شده و متغیر **path** را به عنوان ورودی می‌گیرد. این متغیر رشته ای حاوی آدرس تصویر مد نظر است. سپس عکس را داخل با دستور **open** باز کرده و درون آبجکت **image** میریزیم. همچنین ابزاری تحت عنوان **draw** برای تغییر تصویر ایجاد می‌کنیم. همانطور که در توضیحات آمده این تابع قرار است پیکسل های روشن را به ۱ و پیکسل (تصدانه) های تیره را به ۰ نگاشت دهد.

```
# Determine the image's width and height in pixels.
width = image.size[0]
height = image.size[1]

# Load pixel values for the image.
pix = image.load()

# Define a factor for intensity thresholding.
factor = 100

# Initialize an empty list to store the binary representation.
binary_representation = []
```

در این قسمت از **property** سایز استفاده کرده و طول و عرض تصویر را در متغیرهای مربوطه می‌ریزیم.

همچنین محتوای عددی پیکسل ها را با دستور **load** در متغیر **pix** ذخیره می‌کنیم. متغیر **factor** در واقع مرزی است که مشخص می‌کند کدام پیکسل ها روشن و کدام تیره دسته بندی شود. در خط آخر هم آرایه ای به نام **binary_representation** ایجاد کرده که قرار است محتوای باینری تصویر داخل آن ریخته شود.

```
for i in range(width):
    for j in range(height):
        # Extract the Red, Green, and Blue (RGB) values of the
        pixel.
        red = pix[i, j][0]
        green = pix[i, j][1]
        blue = pix[i, j][2]

        # Calculate the total intensity of the pixel.
        total_intensity = red + green + blue

        # Determine whether the pixel should be white or black
        based on the intensity.
        if total_intensity > ((255 + factor) // 2) * 3:
            red, green, blue = 255, 255, 255 # White pixel
            binary_representation.append(1)
        else:
            red, green, blue = 0, 0, 0 # Black pixel
            binary_representation.append(0)
```

```
# Set the pixel color accordingly.
draw.point((i, j), (red, green, blue))
```

در حلقه‌ی بالا که مرحله‌ی اصلی تبدیل است: دو حلقه‌ی تو در تو برای پیمایش عرضی و طولی تصویر داریم که به ترتیب از سطر یک شروع به پیمایش پیکس ها می‌کند. سپس مقدار **rgb** هر تصدانه را داخل متغیرهای **red, green, blue** ریخته و مجموع آن‌ها را که بیانگر مجموع روشنایی آن تصویر است محاسبه می‌کند. سپس بر حسب این مقدار تصمیم‌گیری انجام می‌شود که آن پیکسل باید سیاه یا سفید شود. هرچه مقدار **rgb** بیشتر باشد پیکسل به رنگ سفید نزدیک تر است و هرچه این مقادیر به صفر نزدیک باشند، پیکسل تیره تر است و به سیاه تبدیل خواهد شد. با مقایسه **total_intensity** با $((255 + factor) // 2) * 3$ ، پیکسل های روشن به سفید تغییر داده شده و به ازای آن مقدار - ۱ وارد لیست باینری می‌شود و برعکس. با ابزار **draw** این تغییر را در تصویر نیز اعمال می‌کنیم.

```
# Clean up the drawing tool.
del draw

# Return the binary representation of the image.
return binary_representation
```

و در نهایت نیز ابزار **draw** را حذف کرده و لیست باینری شده تصویر را بر می‌گردانیم. این تابع تصویر را ساده سازی می‌کند و در واقع می‌تواند پیش‌پردازشی برای تشخیص الگو و تحلیل های پایه ای باشد. همچنین در بسیاری از پردازش های تصویری در یادگیری ماشین این عملیات انجام می‌شود.

توضیح تابع **generateNoisyImages**

```
def generateNoisyImages():
    # List of image file paths
    image_paths = [
        "/content/1.jpg",
        "/content/2.jpg",
        "/content/3.jpg",
        "/content/4.jpg",
        "/content/5.jpg"
    ]

    for i, image_path in enumerate(image_paths, start=1):
        noisy_image_path = f"/content/noisy{i}.jpg"
        getNoisyBinaryImage(image_path, noisy_image_path)
        print(f"Noisy image for {image_path} generated and saved as {noisy_image_path}")
```

این تابع یک حلقه روی تمام عکس هایی که در آرایه `image_paths` آمده می زند و اسم جدید برای تصویر نویزی شده آنها ایجاد و سپس تابع `getNoisyBinaryImage` را برای هر کدام از آنها صدا زده و تصویر نویزی شده آنها را با اسم جدید می سازد.

توضیح تابع `getNoisyBinaryImage`

```
def getNoisyBinaryImage(input_path, output_path):  
    image = Image.open(input_path)  
  
    draw = ImageDraw.Draw(image)  
  
    width = image.size[0]  
    height = image.size[1]  
  
    pix = image.load()  
  
    noise_factor = 100000000
```

این تابع نیز مسیر تصویر خام و مسیری که قرار است تصویر نویزی شده در آن ذخیره شود را به عنوان ورودی می گیرد و همانند تابع باینری، آن تصویر را باز کرده، ابزار رسم را ایجاد کرده، طول و عرض آن را استخراج، همچنین پیکسل های آن را بارگیری می کند. همچنین یک `factor_noise` را نیز مقدار دهی می کند که تعیین کننده شدت نویز اعمالی در تصویر است. در ادامه تاثیر این عامل را می بینیم.

```
for i in range(width):  
    for j in range(height):  
        rand = random.randint(-noise_factor, noise_factor)  
        red = pix[i, j][0] + rand  
        green = pix[i, j][1] + rand  
        blue = pix[i, j][2] + rand  
  
        if red < 0:  
            red = 0  
        if green < 0:  
            green = 0  
        if blue < 0:  
            blue = 0  
        if red > 255:  
            red = 255  
        if green > 255:  
            green = 255  
        if blue > 255:  
            blue = 255
```

```

draw.point((i, j), (red, green, blue))

image.save(output_path, "JPEG")

del draw

generateNoisyImages()

```

سپس در یک حلقه تو در تو روی تمام پیکسل ها حرکت می کند. با استفاده از تابع **rand** یک عدد رندوم در بازه ی (noise_factor, noise_factor-) ایجاد کرده و به تمام رنگ های **rgb** تصدانه اضافه می کند.

بعد با ۶ شرط بررسی می کند که مقادیر **rgb** در بازه ی ۰ تا ۲۵۵ باشند و اگر غیر از این بود، آن ها را در محدوده مجاز قرار می دهد و تغییر را روی پیکسل ها اعمال می کند. در نهایت نیز تصویر تغییر یافته را با اسم جدید ذخیره کرده و ابزار تغییر را حذف می کند.

بهینه کردن کد

به جای ۶ شرط برای چک کردن مرزهای **rgb** از سه خط کد زیر استفاده کردیم:

```

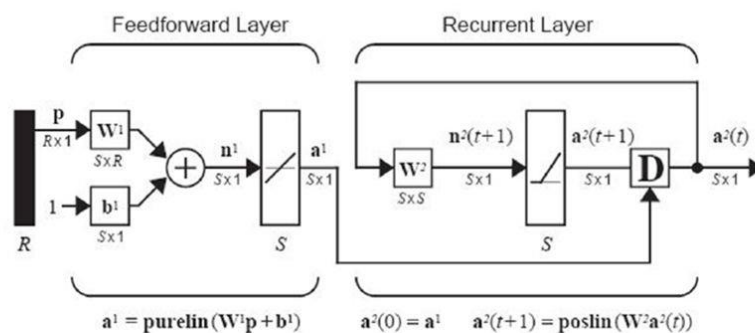
red = max(0, min(255, red))
green = max(0, min(255, green))
blue = max(0, min(255, blue))

```

2

برای مدل خود از شبکه همینگ استفاده کردیم.

Hamming Network



ابتدا تصاویر اصلی که الگوی ما هستند را به شبکه دادیم تا آموزش داده شود. سپس تصاویر نویزی شده را با مقادیر مختلف به شبکه دادیم. در مرحله feedforward ضرایب در ورودی ضرب می‌شوند و سپس این دیتاها در حلقه‌ی maxnet قرار می‌گیرند و پس از تعداد iteration مشخص، نتیجه‌ی مسابقه مشخص می‌شود و کار classification تمام می‌شود.

3

تا ۶۶ درصد missing_point میتواند تمام حروف را به درستی نشان دهد. در درصد‌های بالا ج و د را نمی‌تواند از هم تفکیک کند و ج را نیز تشخیص می‌دهد. ۶۶ درصد هم عدد زیادی است و این قدرت این شبکه را در بازیابی و تشخیص تصاویر از دست رفته نشان می‌دهد.

۴ پرسش چهارشنبه

مدل RBF جواب بهتری داد.

```
<ipython-input-94-4c2dd914f094>:4: UserWarning: Do not pas
super(RBFLayer, self).__init__(**kwargs)
162/162 ----- 0s 2ms/step - loss: 0.4683
RBF Model Loss: 0.46318989992141724
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
162/162 ----- 0s 1ms/step - loss: 0.3117
Dense Model Loss: 0.3086976706981659
```

میگن RBF برای مدل‌های غیر خطی جواب بهتری می‌ده.